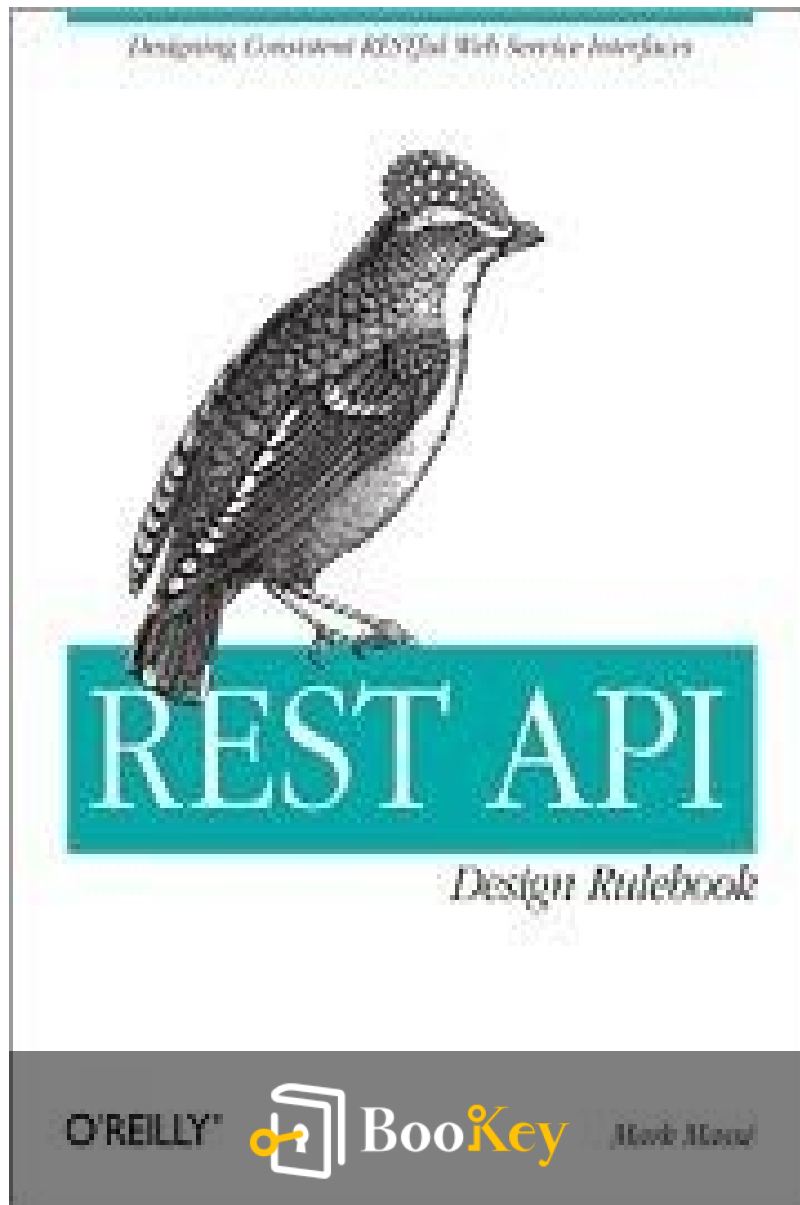


Rest Api Design Rulebook PDF

Mark Masse



More Free Book



Scan to Download



Listen It

Rest Api Design Rulebook

Mastering REST API Design for Effective Web
Services

Written by Bookey

[Check more about Rest Api Design Rulebook Summary](#)

[Listen Rest Api Design Rulebook Audiobook](#)

More Free Book



Scan to Download



[Listen It](#)

About the book

In a competitive landscape where online services vie for user engagement, a well-crafted REST API is essential. This insightful book by Mark Masse offers a collection of API design principles derived from best practices that align with the REST architectural style. Focusing on key elements such as URI design, HTTP usage, media types, and representation formats, readers will discover how to create APIs that adhere to recognized web standards. Masse introduces the Web Resource Modeling Language (WRML), a unique framework designed to simplify the design and implementation of REST APIs. Learn effective strategies for resource addressing, applying HTTP methods and response codes, utilizing metadata through HTTP headers, and accommodating the needs of various client applications, particularly browser-based JavaScript clients. This book emphasizes the importance of designing REST APIs thoughtfully, ensuring they are configured rather than strictly coded.

More Free Book



Scan to Download



Listen It

About the author

Mark Masse is a seasoned expert in software architecture and design, renowned for his extensive contributions to the field of web service and API design. With a rich background in software development and a passion for building robust and scalable solutions, Masse has spent years honing his skills in creating clear and effective RESTful APIs. His insights, drawn from practical experience and industry best practices, have made him a sought-after consultant and speaker. Through his writing, particularly in "REST API Design Rulebook," he has provided invaluable guidance to developers and architects alike, helping them navigate the complexities of modern web services and enable seamless integration across diverse applications.

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

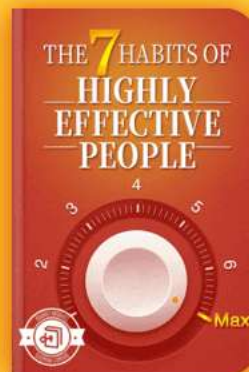
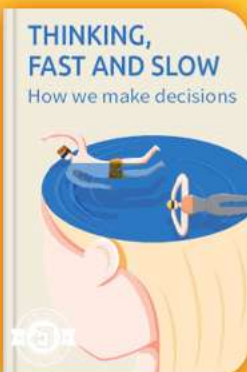


Self-care



Mind & Spirituality

Insights of world best books



Free Trial with Bookey



Summary Content List

Chapter 1 : with URIs

Chapter 2 : with HTTP

Chapter 3 :

Chapter 4 :

Chapter 5 :

Chapter 6 :

Chapter 7 : Appendix. My First REST API

More Free Book



Scan to Download



Listen It

Chapter 1 Summary : with URIs



Identifier Design with URIs

REST APIs utilize Uniform Resource Identifiers (URIs) to identify resources on the web. Effective URI design follows certain principles to ensure clarity and usability.

URI Format Rules

-

Hierarchical Relationships:

Use forward slashes (/) to denote hierarchical relationships between resources.

-

Avoid Trailing Slashes:

More Free Book



Scan to Download



Listen It

Do not include a trailing forward slash at the end of URIs to prevent confusion, as every character contributes to resource identity.

-

Readability:

Use hyphens (-) for better readability instead of underscores (_), which may become obscured in some applications.

-

Lowercase Preference:

Favor lowercase letters in URI paths to avoid case sensitivity issues.

-

No File Extensions:

Avoid using artificial file extensions in URIs, relying instead on media types communicated via headers.

URI Authority Design

-

Consistent Subdomain Names:

Use consistent subdomain names for service identification, such as `api.soccer.restapi.org`.

-

Developer Portal Subdomains:

More Free Book



Scan to Download



Listen It

If a developer portal exists, designate it with a `developer` subdomain for ease of recognition.

Resource Modeling

- Model URIs to correspond to a REST API's resource model with each path segment representing a unique resource.

-

Resource Archetypes:

Define resources using four archetypes: document (singular concept), collection (server-managed directory), store (client-managed repository), and controller (procedural concept).

URI Path Design Rules

-

Document Names:

Use singular nouns for document URIs, e.g.,
`/players/claudio`.

-

Collection Names:

Use plural nouns for collection URIs, e.g., `/players`.

-

More Free Book



Scan to Download



Listen It

Store Names:

Use plural nouns for stores, e.g., `/playlists``.

-

Controller Names:

Use verbs for controller URIs indicating action, e.g., `/students/morgan/register``.

-

Variable Path Segments:

Use identity-based values in URI templates for variable segments (e.g., `{playerId}``) but keep backend identifiers opaque.

URI Query Design Rules

-

Filtering and Pagination:

Utilize the query component to filter collections and paginate results (e.g., `/users?pageSize=25&pageStartIndex=50``).

-

Complex Queries:

For more complicated queries, consider using controller resources to process request bodies instead of relying solely on query parameters.

More Free Book



Scan to Download



Listen It

Recap of Key Terms

-

Authority:

Identifies the governing party of a URI namespace.

-

Collection:

A resource archetype representing a server-management directory.

-

Controller:

Models procedural concepts in API interaction.

-

CRUD:

Refers to fundamental operations—create, retrieve, update, delete.

-

Developer Portal:

A UI for onboarding new clients.

-

Docroot:

The primary resource that serves as the entry point for an API.

More Free Book



Scan to Download



Listen It

-

Document:

Represents a singular resource concept.

-

Query:

The component of a URI used for additional resource identification.

-

Resource Archetypes:

Categories that help describe an API's resources.

This chapter outlines essential principles for designing effective and clear URIs in REST APIs, promoting a structured, understandable resource model.

More Free Book



Scan to Download



[Listen It](#)

Example

Key Point: Effective URI structure enhances usability and clarity for API consumers.

Example: Imagine you are trying to access data for a specific player in a soccer API. You type in the URI `/players/claudio``, a clear and concise path that tells the API exactly what you need: information about Claudio, the player. If the URI were muddled with inconsistent naming or unnecessary trailing slashes, you might end up confused or even unable to access the information you require. Thus, following design principles like using singular nouns for document names and avoiding trailing slashes ensures that you, as a developer or user, can easily navigate the API and retrieve the required resources without any ambiguity.

More Free Book



Scan to Download



Listen It

Request-Line in HTTP request messages, defined by RFC 2616. Each HTTP method has distinct purposes in a REST API's resource model:

-

GET

: Retrieve a representation of a resource's state.

-

HEAD

: Retrieve metadata related to a resource's state.

-

PUT

: Add or update a resource.

-

DELETE

: Remove a resource.

-

POST

: Create a new resource in a collection or execute controllers.

Rules for Request Methods

-

GET and POST Usage

: These methods must not be misused to represent other

More Free Book



Scan to Download



Listen It

request methods—known as tunneling—which undermines HTTP's transparency. Proper use is crucial for effective REST API design.

-

GET Method Specification

: The GET method is essential for retrieving a resource's state in a representational form. GET requests can contain headers but must not have a body. The Web's architecture relies on the GET method's nature, allowing clients to repeat requests safely and caches to serve representations without contacting the server each time.

An example of a GET request using curl to retrieve a "greeting" resource illustrates this process:

```
```bash
$ curl -v http://api.example.restapi.org/greeting
> GET /greeting HTTP/1.1
> User-Agent: curl/7.20.1
> Host: api.example.restapi.org
> Accept: */*
< HTTP/1.1 200 OK
< Date: Sat, 20 Aug 2011 16:02:40 GMT
< Server: Apache
< Expires: Sat, 20 Aug 2011 16:03:40 GMT
```

More Free Book



Scan to Download



Listen It

```
< Cache-Control: max-age=60, must-revalidate
< ETag: text/html:hello world
< Content-Length: 130
< Last-Modified: Sat, 20 Aug 2011 16:02:17 GMT
< Vary: Accept-Encoding
< Content-Type: text/html
<!doctype html><head><meta
charset="utf-8"><title>Greeting</title></head>
<body><div id="greeting">Hello World!</div>
...
```

**More Free Book**



Scan to Download



Listen It

## Example

**Key Point:** Understanding Request Methods

**Example:** When working with REST APIs, mastering HTTP request methods empowers you to effectively interact with resources. For example, imagine you're developing a mobile app that fetches user profiles.

When a user opens the app, you utilize a GET request to the endpoint ``http://api.example.restapi.org/users/1234`` to retrieve the profile of user '1234'. By properly leveraging the GET method, you ensure users receive the latest profile data, enabling a smooth interaction experience as they scroll through detailed information on their mobile screens.

More Free Book



Scan to Download



Listen It

## Critical Thinking

**Key Point:** Request Method Integrity

**Critical Interpretation:** The chapter emphasizes the importance of correctly utilizing HTTP request methods like GET and POST in REST APIs, which is vital for the architecture's reliability. Misuse, such as tunneling, can compromise clarity and functionality, suggesting that adherence to standards is crucial. However, critics argue that strict adherence can sometimes stifle innovation and flexibility in API design, as evidenced in discussions on platforms like Stack Overflow and various REST API design forums.

More Free Book



Scan to Download



Listen It

# Chapter 3 Summary :

## Chapter 4: Metadata Design

### HTTP Headers Overview

This chapter discusses the various forms of metadata conveyed through HTTP headers in REST API design. It outlines standard headers for resource information, representation indication, and cache control.

### Rules for HTTP Headers

-

#### **Content-Type Must Be Used**

: Indicates the data type in a request/response body, crucial for processing messages.

-

#### **Content-Length Should Be Used**

: Specifies the size of the entity-body, aiding clients in verifying the completeness of received data and making HEAD requests.

More Free Book



Scan to Download



Listen It

-

### **Last-Modified Should Be Used in Responses**

: Provides a timestamp for when a resource last changed, helping clients and caches evaluate resource freshness.

-

### **ETag Should Be Used in Responses**

: An opaque string representing the version of the resource, enabling clients to perform conditional requests.

-

### **Stores Must Support Conditional PUT Requests**

: Clients express intent to insert or update resources through conditional headers (If-Unmodified-Since and If-Match).

-

### **Location Must Be Used**

: Indicates the URI of a newly created resource in responses, especially after successful resource creation.

-

**Cache-Control, Expires, and Date Response**

**Install Bookey App to Unlock Full Text and Audio**

**More Free Book**



Scan to Download



Listen It



Scan to Download



# Why Bookey is must have App for Book Lovers



## 30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



## Text and Audio format

Absorb knowledge even in fragmented time.



## Quiz

Check whether you have mastered what you just learned.



## And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



# Chapter 4 Summary :

Section	Key Points
Representation Design	
Message Body Format	JSON should be supported for resource representation.   JSON must be well-formed.   XML and other formats may optionally be used.   No additional envelopes for resource state.
Hypermedia Representation	Consistent link representation.   Consistent link relations.   Consistent links structure.   Self link inclusion.   Minimize entry point URIs.   State-sensitive advertising of actions.
Media Type Representation	Consistent media type formats.   Consistent media type schemas.   Field and Constraint Representations.   Link Aspects.
Error Representation	Error form consistency.   Error response structure.   Common error types.
Recap	The chapter emphasizes design rules for resource representations, focusing on JSON usage, uniform link representation, error handling clarity, and media type structuring.

## Representation Design

More Free Book



Scan to Download



Listen It

## Message Body Format

A REST API often uses the entity body of a response message to convey the state of the identified resource using text-based formats. JSON and XML are the most popular formats for this representation. JSON, preferred in this chapter, features a simple structure with curly brackets for organization. Unlike JSON, XML uses matching tag pairs, which can complicate formatting. Key rules for representation are outlined:

1.

### **JSON should be supported for resource representation**

: JSON is favored for data exchange due to its lightweight nature and ease of integration with browsers. If no standard format exists for a resource type, JSON should be used.

2.

### **JSON must be well-formed**

: JSON requires strict formatting, such as enclosing names in double quotes.

3.

### **XML and other formats may optionally be used**

More Free Book



Scan to Download



Listen It

: While JSON should be a primary representation, other formats like XML can be supported if clients specify their preferences.

4.

### **No additional envelopes**

: The resource state should be presented directly without additional transport wrappers.

## **Hypermedia Representation**

REST APIs utilize hypermedia to convey relationships between resources within representational bodies. Key rules include:

1.

### **Consistent link representation**

: Links should follow a uniform format within resource representations. Each link includes fields like ``href``, ``rel``, and optional types.

2.

### **Consistent link relations**

: Each link's relationship is defined through a designated structure that informs clients on how to interact with various links.

3.

**More Free Book**



Scan to Download



Listen It

## **Consistent links structure**

: A top-level name-value pair in JSON that groups available links for easier discovery.

4.

## **Self link inclusion**

: Each resource representation should include a self link, indicating a reference to itself.

5.

## **Minimize entry point URIs**

: Documentation should focus on a central "docroot" URI, rather than exposing many individual resource URIs.

6.

## **State-sensitive advertising of actions**

: Links must reflect the state of resources and available actions, allowing for context-sensitive interactions.

## **Media Type Representation**

Media types allow APIs to describe the format and structure of their content. Key rules include:

1.

## **Consistent media type formats**

: A uniform representation should describe media type formats.

**More Free Book**



Scan to Download



Listen It

2.

### **Consistent media type schemas**

: Separate schema resources abstract the structure of representations, aiding client understanding.

3.

### **Field and Constraint Representations**

: Definitions for field types and constraints that govern the values a field can take.

4.

### **Link Aspects**

: Link formulas can associate the presence of links with conditions derived from state facts.

## **Error Representation**

APIs should provide consistent forms for errors in response messages:

1.

### **Error form consistency**

: Each individual error should follow a defined format.

2.

### **Error response structure**

: A structured response for error situations should be adopted, including an appropriate media type.

**More Free Book**



Scan to Download



Listen It

3.

### **Common error types**

: Shared error types can facilitate uniformity across multiple APIs by defining base error types.

### **Recap**

The chapter emphasizes critical design rules for resource representations, ensuring that they are coherent, easily usable, and contextually aware in their interactions and responses. It highlights the importance of JSON, uniform link representation, clarity in error handling, and the structuring of media types.

**More Free Book**



Scan to Download



Listen It

## Critical Thinking

**Key Point:** The Importance of JSON for Resource Representation

**Critical Interpretation:** While the chapter underscores JSON's advantages for REST APIs, emphasizing its lightweight and browser-friendly nature, it's essential to recognize that this preference may overlook scenarios where XML or other formats could provide better functionality or compatibility. Critics argue that adhering strictly to JSON may inadvertently limit the flexibility and interoperability of APIs in diverse environments (Bishop, R. A. (2011). 'HTTP and REST: A Primer. Computer Communications Journal').

More Free Book



Scan to Download



Listen It

# Chapter 5 Summary :

## Client Concerns

### Introduction

This chapter discusses how REST APIs cater to their clients, which include various applications like web scripts and critical business software. It offers design principles focused on common client concerns and rules for JavaScript clients.

### Versioning

-

#### **New URIs for New Concepts:**

When introducing new concepts, new URIs should be created without embedding version information, as URIs should consistently identify the same semantic model.

-

#### **Use of Schemas for Representational Form Versions:**

Versioned schema documents help manage changes in the resource's representation while maintaining backward

**More Free Book**



Scan to Download



Listen It

compatibility.

-

### **Entity Tags for Representational State Versions:**

ETags are utilized to maintain the versioning of a resource's state.

## **Security**

-

### **OAuth for Resource Protection:**

OAuth offers a standardized method for secure authorization, enabling users to share resources without compromising credentials.

-

### **API Management Solutions for Resource Security:**

Network-based intermediaries, like API reverse proxies, can help secure REST APIs, often supporting OAuth and other protocols.

## **Response Representation Composition**

-

### **Use of Query Component for Partial Responses:**

Clients can specify partial responses by using the fields

**More Free Book**



Scan to Download



Listen It

query parameter to trim unnecessary data from the API response.

-

### **Embedding Linked Resources:**

Clients can use the embed query parameter to determine how linked resources are represented within the response, enhancing flexibility.

### **Processing Hypermedia**

Clients interact with REST API responses by following specified hypermedia structures, allowing processed interactions based on links and request methods.

### **JavaScript Clients**

JavaScript clients, due to the same origin policy, may require special considerations:

-

### **Support for JSONP:**

JSONP enables cross-origin GET requests by dynamically creating script tags for data retrieval.

-

### **Support for CORS:**

**More Free Book**



Scan to Download



Listen It

CORS standardizes cross-origin requests, allowing multiple HTTP methods beyond just GET, improving overall API usability.

## Recap

The chapter reinforces various concepts such as API reverse proxies, CORS, embedded links, exclusion and inclusion lists, JSONP, mashups, OAuth, partial responses, and the same origin policy, summarizing their relevance for REST API design.

**More Free Book**



Scan to Download



Listen It

# Chapter 6 Summary :

## CHAPTER 7 Final Thoughts

### State of the Art

Implementing REST APIs today poses several challenges due to the limitations of existing development tools and frameworks. Many frameworks designed for REST API development stem from web application origins, causing an oversimplification of REST APIs. While these frameworks support routing and serialization, they often lack essential features such as resource model separation, hypermedia structures, HATEOAS automation, and media type schema validation. Consequently, developers face the difficult choice of either omitting features or developing them independently, leading some to rely on API management solutions—though this can risk vendor lock-in.

### Uniform Implementation

Coding a REST API often feels inconsistent, and a better

**More Free Book**



Scan to Download



Listen It

approach is proposed where REST APIs should be designed, not coded. The WRML (Web REST Modeling Language) conceptual framework suggests a uniform REST API layer that focuses on configuration-driven processing instead of repetitive coding. This would involve creating an engine within a web resource server that manages request handling through a series of configurable steps, allowing for a more standardized approach to REST API design.

## Consistency Benefits

Uniformity across REST APIs can drive improvements in efficiency and ease of use, similar to how consistent HTML, CSS, and JavaScript implementations benefited web development. By reducing variability in REST API designs, server workloads can decrease as client devices take on more processing tasks, ultimately minimizing operational costs. This harmonized architecture allows backend developers to

**Install Bookey App to Unlock Full Text and Audio**

More Free Book



Scan to Download



Listen It

Ad



Scan to Download



App Store  
Editors' Choice



22k 5 star review

## Positive feedback

Sara Scholz

...tes after each book summary  
...erstanding but also make the  
...and engaging. Bookey has  
...ding for me.

**Fantastic!!!**



I'm amazed by the variety of books and languages  
Bookey supports. It's not just an app, it's a gateway  
to global knowledge. Plus, earning points for charity  
is a big plus!

Masood El Toure

Fi



Ab  
bo  
to  
my

José Botín

...ding habit  
...o's design  
...ual growth

**Love it!**



Bookey offers me time to go through the  
important parts of a book. It also gives me enough  
idea whether or not I should purchase the whole  
book version or not! It is easy to use!

Wonnie Tappkx

**Time saver!**



Bookey is my go-to app for  
summaries are concise, ins  
curated. It's like having acc  
right at my fingertips!

**Awesome app!**



I love audiobooks but don't always have time to listen  
to the entire book! bookey allows me to get a summary  
of the highlights of the book I'm interested in!!! What a  
great concept !!!highly recommended!

Rahul Malviya

**Beautiful App**



This app is a lifesaver for book lovers with  
busy schedules. The summaries are spot  
on, and the mind maps help reinforce wh  
I've learned. Highly recommend!

Alex Walk

Free Trial with Bookey



# Chapter 7 Summary : Appendix. My First REST API

## APPENDIX My First REST API

### Introduction

In June 1997, Mark Masse designed his first REST API while working at Starwave, focusing on developing a Java applet for the NASCAR Online website.

### Project Overview

Masse's task was to create the Race Tracker applet, which displayed the status of lead cars during live races. The applet required continuous data fetching from a server, with HTTP being the viable option for data transmission over the Internet.

### Server-Side Implementation

**More Free Book**



Scan to Download



Listen It

The server side utilized a proprietary system named “Bulldog” for automated real-time data processing and publishing. This system managed the data from wire feeds, created Java objects, and published them through a web page template, resulting in unique URIs for each resource.

## **Data Representation**

The applet used HTTP GET requests to periodically fetch resource representations, which were formatted as plain text files. Each data element was organized in rows, with attributes delimited by the pipe character. This representation included hyperlinks for additional resources, such as driver images.

## **Conclusion**

Though the Race Tracker applet's Web API did not strictly follow all REST rules, it utilized key elements like URIs, HTTP, and hypermedia representations. This exemplifies that REST is not a new concept but rather a reflection of existing web practices.

## **About the Author**

**More Free Book**



Scan to Download



**Listen It**

Mark Massé is a Senior Director of Engineering at ESPN, based in Seattle. He enjoys family time, soccer, the Seattle Sounders, and discussions about video games in his spare time, especially with his partner, Shawna.

**More Free Book**



Scan to Download



**Listen It**



# Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

## The Concept



×



×



This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

## The Rule



Earn 100 points



Redeem a book



Donate to Africa

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey



# Best Quotes from Rest Api Design Rulebook by Mark Masse with Page Numbers

[View on Bookey Website and Generate Beautiful Quote Images](#)

## Chapter 1 | Quotes From Pages -40

- 1.The only thing you can use an identifier for is to refer to an object. When you are not dereferencing, you should not look at the contents of the URI string to gain other information."  
—Tim Berners-Lee
- 2.A forward slash (/) character is used in the path portion of the URI to indicate a hierarchical relationship between resources.
- 3.To make your URIs easy for people to scan and interpret, use the hyphen (-) character to improve the readability of names in long path segments.
- 4.Lowercase letters should be preferred in URI paths since capital letters can sometimes cause problems.
- 5.URIs should not be used to indicate that a CRUD function

More Free Book



Scan to Download



[Listen It](#)

is performed. URIs should be used to uniquely identify resources.

## **Chapter 2 | Quotes From Pages -52**

1. A REST API must not compromise its design by misusing HTTP's request methods in an effort to accommodate clients with limited HTTP vocabulary.
2. Clients count on being able to repeat GET requests without causing side effects.
3. Always make proper use of the HTTP methods as specified by the rules in this section.
4. GET must be used to retrieve a representation of a resource.

## **Chapter 3 | Quotes From Pages -64**

1. The Content-Type header names the type of data found within a request or response message's body.
2. The Last-Modified header applies to response messages only.

**More Free Book**



Scan to Download



Listen It

3. A REST API must rely on the client to include the If-Unmodified-Since and/or If-Match request headers to express their intent.
4. Location must be used to specify the URI of a newly created resource.
5. Caching is one of the most useful features built on top of HTTP.
6. Custom HTTP headers must not be used to change the behavior of HTTP methods.
7. A consistent form should be used to represent media type formats.
8. REST APIs should not do this unless absolutely necessary.
9. Media type negotiation should be supported when multiple representations are available.
10. The future-proof nature of this design may prove especially useful when a REST API wishes to adopt a new format that is not yet widely supported by its clients.





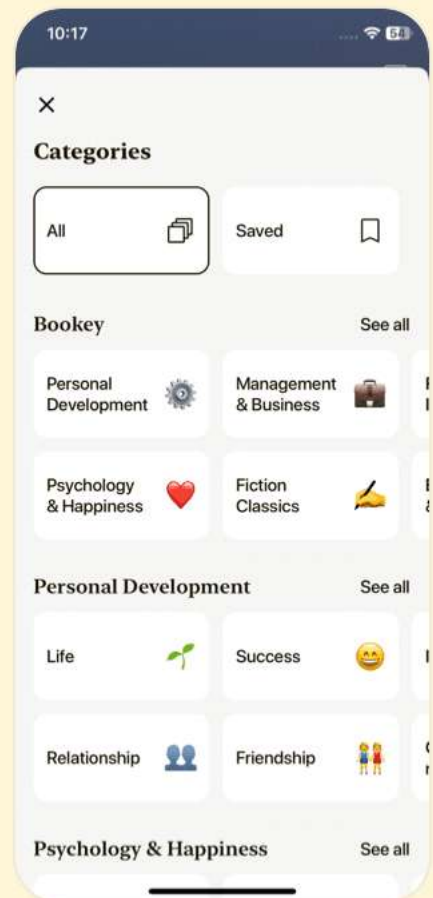
Download Bookey App to enjoy

**1 Million+ Quotes**

**1000+ Book Summaries**

**Free Trial Available!**

Scan to Download



## Chapter 4 | Quotes From Pages -88

- 1.Rule: JSON should be supported for resource representation
- 2.Rule: A consistent form should be used to represent links
- 3.Rule: Links should be used to advertise a resource's available actions in a state-sensitive manner
- 4.Rule: A self link should be included in response message body representations
- 5.Rule: A consistent form should be used to represent errors

## Chapter 5 | Quotes From Pages -102

- 1.New URIs should be used to introduce new concepts.
- 2.Schemas should be used to manage representational form versions.
- 3.Entity tags should be used to manage representational state versions.
- 4.OAuth may be used to protect resources.
- 5.API management solutions may be used to protect resources.



- 6.The query component of a URI should be used to support partial responses.
- 7.JavaScript clients should be supported to provide multi-origin read access from JavaScript.
- 8.CORS should be supported to provide multi-origin read/write access from JavaScript.

## **Chapter 6 | Quotes From Pages -110**

- 1.Today, implementing our REST API designs is harder than it ought to be.
- 2.Coding a REST API has never felt right to me. I believe that REST APIs should be designed and configured, but not coded.
- 3.REST APIs, while becoming ubiquitous, are far from uniformly designed.
- 4.The greatest benefit of a standardized design and implementation methodology is the availability of helpful frameworks and tools that increase developer productivity.
- 5.In a world of multi-device clients with different formatting needs, this architecture frees up server developers to focus



on advancing the web application's business logic in their backend systems.

**More Free Book**



Scan to Download



**Listen It**



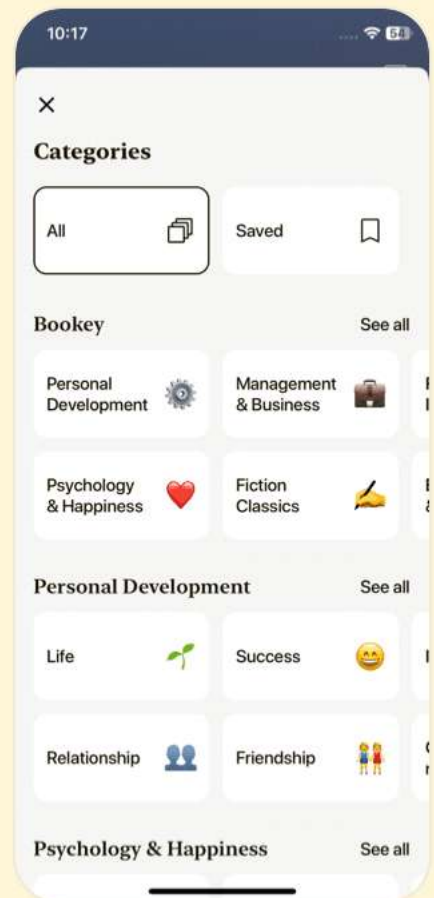
Download Bookey App to enjoy

**1 Million+ Quotes**

**1000+ Book Summaries**

**Free Trial Available!**

Scan to Download



## Chapter 7 | Quotes From Pages 111-114

1.REST describes the way the Web already works.

REST isn't an invention; it is a prescription.

2.the resources were uniquely identified with its own URI.

3.the applet needed to talk to a Web API.

More Free Book



Scan to Download



Listen It



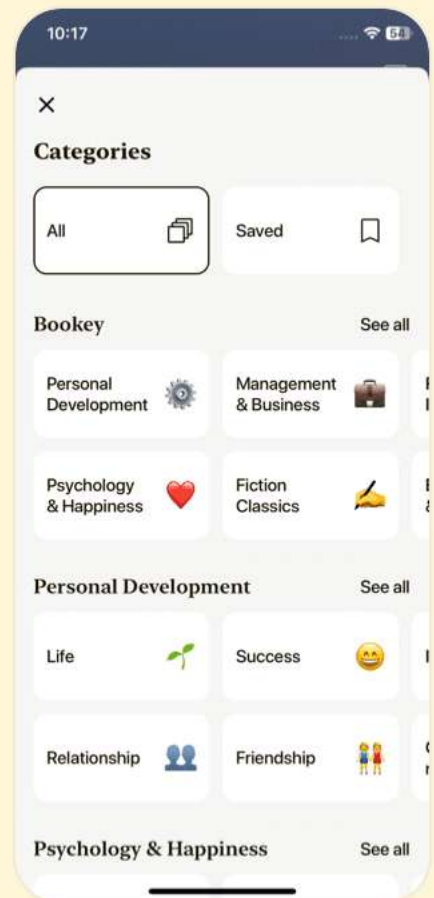
Download Bookey App to enjoy

**1 Million+ Quotes**

**1000+ Book Summaries**

**Free Trial Available!**

Scan to Download



# Rest Api Design Rulebook Questions

[View on Bookey Website](#)

## Chapter 1 | with URIs| Q&A

### 1.Question

**What is the importance of URI design in REST APIs?**

Answer:URI design plays a critical role in REST APIs as it defines how resources are identified and accessed over the web. Good URI design improves the clarity and usability of an API, making it easier for client developers to understand and navigate the API's resource model.

### 2.Question

**Why should forward slashes (/) be used to indicate hierarchical relationships in URIs?**

Answer:Forward slashes are used to separate path segments in a URI, clearly defining the hierarchical structure of resources. This provides both the server and clients with an intuitive way to understand resource relationships.

### 3.Question

More Free Book



Scan to Download



[Listen It](#)

## **What are the implications of including a trailing forward slash (/) in a URI?**

Answer: Including a trailing slash may lead to confusion, as different URIs (with and without the trailing slash) may be treated as distinct resources. Therefore, it's best practice to avoid trailing slashes to maintain URI uniqueness and avoid semantic misunderstanding.

### **4.Question**

## **How do hyphens improve the readability of URIs?**

Answer: Hyphens serve as visible separators that enhance readability in URIs, especially in longer path segments. Using hyphens instead of spaces improves the ease with which users can scan and interpret URIs.

### **5.Question**

## **Why are underscores (\_) discouraged in URIs?**

Answer: Underscores can be visually obscured due to underlining by browsers and editors, making them less clear. In contrast, hyphens are more visible and thus preferred for clarity.

**More Free Book**



Scan to Download



**Listen It**

## 6.Question

**Explain the significance of using lowercase letters in URI paths.**

Answer:Lowercase letters are preferred because URIs are case-sensitive, and using lowercase avoids potential confusion resulting from mixed-case variations that could lead to different resource mappings.

## 7.Question

**What is the reason for avoiding file extensions in URIs?**

Answer:Excluding file extensions from URIs prevents the API from being tied to specific formats. Instead, REST APIs utilize media types specified in headers, promoting a cleaner and more flexible API design.

## 8.Question

**What does resource modeling signify in the context of REST APIs?**

Answer:Resource modeling establishes the key concepts of an API, helping developers define the structure of API resources systematically, much like designing a database schema.

More Free Book



Scan to Download



Listen It

## 9.Question

**What are the four distinct resource archetypes in REST APIs?**

Answer:The four archetypes are: Document (single instances), Collection (directories of resources), Store (client-managed repositories), and Controller (procedural functions for actions). Each archetype helps maintain clarity and organization within the API.

## 10.Question

**What should be used to identify document resources in URIs?**

Answer:Document resources should be represented using singular nouns to signify individual items, enhancing the clarity of what the URI points to.

## 11.Question

**How should collections and stores be named in URIs?**

Answer:Collections should use plural nouns to reflect their contents, while stores should also be named with plural nouns, enabling the identification of groups of resources.

## 12.Question

**More Free Book**



Scan to Download



Listen It

## **Why is CRUD terminology avoided in URI design?**

Answer: CRUD terminology should not be embedded in URIs to maintain the purpose of URIs as resource identifiers.

Instead, HTTP request methods should handle the CRUD operations, keeping the URIs focused solely on identifying resources.

### **13.Question**

## **What is one way URI queries can enhance the functionality of REST APIs?**

Answer: URI queries can be used to filter and paginate results, allowing clients to retrieve specific subsets of resources based on defined criteria, thereby improving API usability.

### **14.Question**

## **What does treating URIs as opaque identifiers entail?**

Answer: Treating URIs as opaque identifiers means that the structure of the URI should not reveal information about the resource. This confines clients to referring to the resource solely by its URI without needing to understand its

**More Free Book**



Scan to Download



Listen It

composition.

### 15.Question

**What potential issue arises from using different casing in URIs?**

Answer:Using different casing in URIs can lead to confusion as URIs are case-sensitive, possibly resulting in clients referring to completely different resources.

### 16.Question

**Why is it necessary to maintain a clean URI structure?**

Answer:A clean URI structure aids in clarity, reduces complexity, and ensures that users and client applications can easily acquire, interpret, and use resources without misunderstanding.

### 17.Question

**How do variable path segments improve URI designs?**

Answer:Variable path segments allow URIs to be dynamic and adaptable, enabling APIs to generate unique identifiers that are meaningful to the clients while maintaining a clear structure.

### 18.Question

More Free Book



Scan to Download



Listen It

## **What role does pagination play in URI queries for REST APIs?**

Answer:Pagination through URI queries helps manage and limit the amount of data returned per request, enhancing performance and user experience by allowing clients to retrieve data in manageable chunks.

## **Chapter 2 | with HTTP| Q&A**

### **1.Question**

#### **What is the significance of using the correct HTTP request methods in a REST API design?**

Answer:Using the correct HTTP request methods ensures that the API functions as intended and adheres to the principles of REST. For instance, the GET method should be solely used for retrieval without side effects, while POST is meant for creating new resources. This clarity prevents misinterpretation and misuse of the API, which can lead to unpredictable behavior and errors.

### **2.Question**

**More Free Book**



Scan to Download



Listen It

## **How does the GET method maintain the integrity of web communication?**

Answer: The GET method allows clients to request the state of a resource consistently and without side effects. This functionality is crucial because it enables caching mechanisms to serve stored responses independently, enhancing efficiency and speed of web communications.

### **3.Question**

## **Can you explain the consequences of 'tunneling' in HTTP requests?**

Answer: Tunneling occurs when an HTTP method is misused, such as using GET to perform actions that are meant for POST or DELETE. This undermines the transparency and intent of the protocol, leading to confusion, errors in resource manipulation, and potential security vulnerabilities in the API.

### **4.Question**

## **Why might developers prefer using curl over a web browser for testing REST APIs?**

**More Free Book**



Scan to Download



Listen It

Answer: Developers prefer curl because it provides direct access to the full range of HTTP features, is scriptable, and allows for precise control over request parameters. This makes it easier to automate testing and handle more complex interactions with REST APIs compared to the limitations of a standard web browser.

### 5.Question

**What are some key features of the Request-Line in an HTTP request message?**

Answer: The Request-Line is a critical part of the HTTP request format, consisting of the request method (like GET or POST), the Request-URI for the resource, and the HTTP version. This structure is designed to clearly communicate the client's intent regarding how to interact with the resource.

### 6.Question

**Why is the proper use of the HEAD method important in a REST API?**

Answer: The HEAD method is used to retrieve metadata about a resource without transferring the resource's body.

More Free Book



Scan to Download



Listen It

This is important for checking the status and properties of a resource without incurring the overhead of downloading the full representation, which is useful for resource management and optimization.

## 7.Question

**What is an example of a GET request using curl, and what does it return?**

Answer:An example of a GET request using curl is:

```
```bash
$ curl -v http://api.example.restapi.org/greeting
```
```

This command retrieves the current state of the 'greeting' resource, typically returning an HTML page that includes the greeting message, e.g., 'Hello World!'. The response also includes headers like cache control and the response status to help the client understand how to handle the data.

## Chapter 3 | Q&A

### 1.Question

More Free Book



Scan to Download



Listen It

## **What is the importance of the Content-Type header in a REST API?**

Answer: The Content-Type header is crucial as it tells clients and servers how to interpret the data contained within the request or response body. For instance, if a REST API sends a response with Content-Type: application/json, the client knows to expect and process the data as JSON, ensuring correct parsing and handling.

### **2.Question**

## **Why should the Content-Length header be used in responses?**

Answer: The Content-Length header indicates the size of the entity body in bytes, which is important for clients to validate that they have received the entire response. It can also enable clients to make HEAD requests to determine the size of resources before downloading them.

### **3.Question**

## **What role does the Last-Modified header play in REST APIs?**

**More Free Book**



Scan to Download



Listen It

Answer: The Last-Modified header provides a timestamp indicating when the resource's representation was last changed. This allows clients and caching intermediaries to determine whether they need to fetch a new copy of the resource, promoting efficient use of bandwidth.

#### 4.Question

**How does the ETag header enhance caching efficiency?**

Answer: The ETag header carries an opaque identifier for a specific version of a resource. When clients use this ETag in subsequent requests via the If-None-Match header, they can prevent unnecessary data transfers. If the resource has not changed, the server can respond with a 304 Not Modified status instead of sending the full data again.

#### 5.Question

**Why is it necessary to support conditional PUT requests?**

Answer: Conditional PUT requests allow clients to specify their intent to update a resource only if it matches the current representation on the server. This prevents unintended overwrites and conflicts, enhancing data integrity and

More Free Book



Scan to Download



Listen It

collaboration among multiple clients interacting with the same resource.

### 6.Question

**What is the significance of the Location header in a REST API?**

Answer:The Location header is vital when a new resource is created. It provides the URI of the newly created resource, allowing clients to reference it directly in future interactions.

### 7.Question

**How do Cache-Control, Expires, and Date headers work together in caching responses?**

Answer:These headers help manage the caching strategy for responses. Cache-Control sets a maximum freshness duration, Expires indicates a specific expiration time, and Date provides the current time of the response. Together, they guide clients and caches in deciding when a response may be stale and should be refreshed.

### 8.Question

**What is the purpose of media type negotiation in REST APIs?**

More Free Book



Scan to Download



Listen It

Answer:Media type negotiation allows clients to specify their desired response format through the Accept header. This enables REST APIs to serve different representations of the same resource, adapting to the client's needs efficiently.

### 9.Question

**What are vendor-specific media types and why are they useful?**

Answer:Vendor-specific media types utilize the 'vnd' prefix to indicate ownership by a specific organization, thereby providing clear content descriptions that enhance interoperability and convey semantic meaning, which is especially useful in complex applications.

### 10.Question

**Why should custom HTTP headers be avoided for changing the behavior of HTTP methods?**

Answer:Custom headers can lead to unreliable behavior if clients and servers are not designed to handle them correctly. Instead, critical information should be conveyed using standard headers or in the request/response body to maintain



compatibility and clarity.

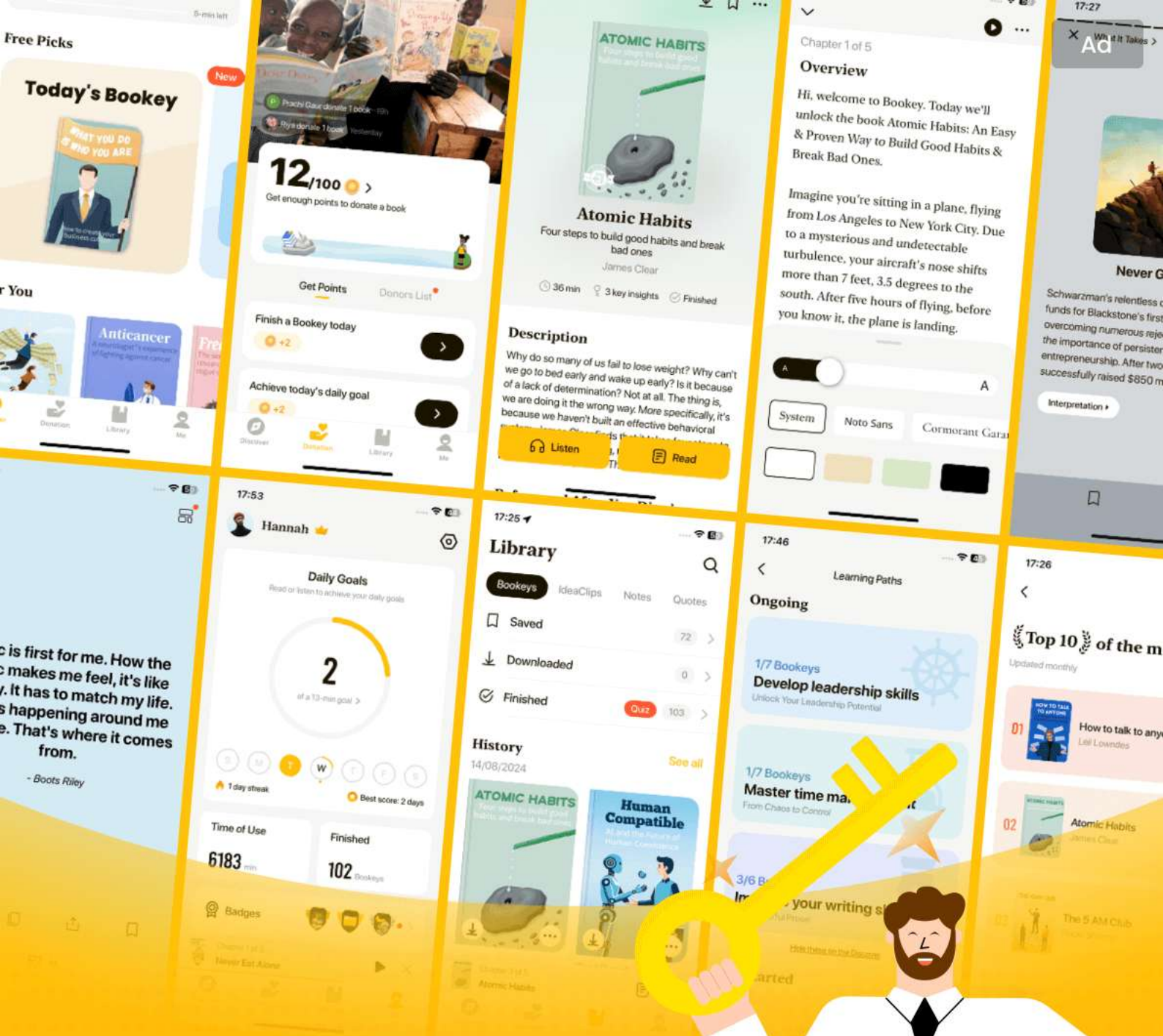
**More Free Book**



Scan to Download



**Listen It**



# World's best ideas unlock your potential

Free Trial with Bookey



Scan to download



## Chapter 4 | Q&A

### 1.Question

**What format is commonly used for resource representation in REST APIs, and what are its advantages?**

Answer:JSON is the most commonly used format for resource representation in REST APIs due to its lightweight nature and ease of integration with web technologies, particularly JavaScript. It provides a clear and concise way to organize data using key-value pairs, making it accessible for both developers and systems.

### 2.Question

**Why is it important for JSON to be well-formed in REST APIs?**

Answer:Well-formed JSON is crucial because it ensures that the data structure is valid and can be consistently parsed by different clients and servers. A malformed JSON can lead to errors in data handling and communication between services, thus affecting API reliability.

More Free Book



Scan to Download



Listen It

### 3.Question

**How do links in REST API responses enhance client interactions?**

Answer:Links in REST API responses serve as actionable references that guide clients on what operations can be performed on a resource. They enable clients to navigate through related resources, thus providing a richer and dynamic interaction experience.

### 4.Question

**What is the role of media type negotiation in REST APIs?**

Answer:Media type negotiation allows clients to specify the format they prefer for responses, ensuring that the data is delivered in a form that best suits their needs. This enhances interoperability and user experience across different platforms.

### 5.Question

**Why should a REST API minimize the number of advertised entry point URIs?**

Answer:Minimizing advertised entry points helps prevent tightly coupled clients that rely on specific URIs, making the

More Free Book



Scan to Download



Listen It

API more flexible and easier to evolve without breaking existing client implementations.

### 6.Question

**What are state facts and how do they relate to link formulas in a REST API?**

Answer:State facts are Boolean variables that represent specific conditions within a resource. They are used in link formulas to determine the availability of hypermedia links in API responses, enabling adaptive and context-sensitive navigation.

### 7.Question

**How should error responses be structured in a REST API, and why is this important?**

Answer>Error responses should follow a consistent JSON structure that includes an error ID and a description. This allows clients to programmatically identify and understand errors, enhancing error handling and user experience.

### 8.Question

**What is the advantage of having a consistent structure for links in REST API representations?**

More Free Book



Scan to Download



Listen It

Answer:A consistent structure for links enables clients to predictably locate and interact with hyperlinks, facilitating automated processing and reducing the learning curve for developers integrating with the API.

### 9.Question

**How does the self link enhance resource representation in REST APIs?**

Answer:The self link provides a reference to the resource being represented, allowing clients to easily retrieve or manipulate that resource without needing to be aware of its URI beforehand.

### 10.Question

**What is the significance of error schema sharing across different APIs?**

Answer:Sharing error schemas allows for standardization in error handling across different APIs. This reduces redundancy and ensures that client developers can handle errors consistently, leading to a more robust and understandable API ecosystem.

More Free Book



Scan to Download



Listen It

## Chapter 5 | Q&A

### 1.Question

**What is the significance of using new URIs to introduce new concepts in a REST API?**

Answer:New URIs should be introduced when exposing new concepts to maintain clarity and consistency. This ensures that each URI uniquely identifies a specific resource and prevents confusion over versioning. The identity of a resource must remain constant over time, despite changes in its representation or state.

### 2.Question

**How can a REST API manage representational form versions without breaking backward compatibility?**

Answer:By using versioned schema documents and allowing clients to negotiate media types, REST APIs can add fields and links to new schema versions. This method enables the introduction of new features while ensuring that existing clients continue to operate without issue.

More Free Book



Scan to Download



Listen It

### 3.Question

**How does OAuth enhance the security of REST APIs?**

Answer:OAuth provides a standardized method for secure authorization, allowing users to share resources without exposing sensitive credentials. It validates a client's request for accessing protected resources, facilitating controlled access based on user permissions.

### 4.Question

**In what way can the query component of a URI be used to enhance client response customization?**

Answer:Clients can manipulate the query component to request only relevant or necessary data from a REST API. This can be done using parameters for partial responses, allowing clients to specify which fields they want included in the response, thereby optimizing bandwidth and response times.

### 5.Question

**What is JSONP and how does it allow cross-origin requests?**

Answer:JSONP (JSON with Padding) is a technique that

More Free Book



Scan to Download



Listen It

allows JavaScript clients to make cross-origin GET requests by injecting a script element into the DOM. This method circumvents browser restrictions on XMLHttpRequest for third-party domains, enabling the API to return data wrapped in a callback function specified by the client.

## 6.Question

**What role does CORS play in REST API interactions with JavaScript clients?**

Answer:CORS (Cross-Origin Resource Sharing) standardizes cross-origin requests in web browsers, allowing REST APIs to declare permitted origins via HTTP headers. It enables JavaScript clients to access resources across different origins with more flexibility than JSONP, supporting a variety of HTTP methods.

## 7.Question

**How does the exclusion list function within a REST API's response?**

Answer:An exclusion list allows clients to specify fields that should be omitted from the response. This customization

More Free Book



Scan to Download



Listen It

improves efficiency by reducing the amount of data transferred, ensuring clients receive only the necessary information.

### 8.Question

**Why is it important for REST APIs to support the same origin policy in browser-based JavaScript clients?**

Answer:Supporting the same origin policy enhances security by restricting JavaScript clients from accessing resources on different origins, protecting sensitive user data from potential leaks or unauthorized access.

### 9.Question

**What is the value of response representation composition for REST API clients?**

Answer:Response representation composition allows clients to customize the data they receive, improving usability and efficiency. By controlling field inclusion, exclusion, and linked resource embedding, clients can tailor responses to fit their specific needs and reduce overhead.

### 10.Question

**How can REST APIs show respect for client needs in**

More Free Book



Scan to Download



Listen It

## **response representations?**

Answer:By providing options to customize response representations through URI query parameters for field selection and resource embedding, REST APIs can accommodate various client requirements, enhancing the overall client experience.

## **Chapter 6 | Q&A**

### **1.Question**

**Why is the implementation of REST APIs considered harder than it should be?**

Answer:The implementation is harder than necessary due to the lack of adequate tools and frameworks that are specifically designed for REST API development. Many existing frameworks are adaptations of web application frameworks, which do not align perfectly with the REST architectural style. Additionally, these frameworks often overlook essential features like natural resource separation, hypermedia structure support, automated

**More Free Book**



Scan to Download



Listen It

HATEOAS implementations, and integration with variable client requirements.

## 2.Question

**What are some core features lacking in current REST API development frameworks?**

Answer:Current frameworks typically lack support for: natural separation of resource models from the server implementation, uniform hypermedia structures, automated HATEOAS, media type schema validation, dynamic response bodies, integration with client identification, and multi-origin resource sharing.

## 3.Question

**What is the impact of relying too much on API management solutions?**

Answer:Over-reliance on API management solutions can lead organizations to become dependent on specific vendor implementations, which complicates switching vendors or frameworks and reduces flexibility. This could stifle innovation and adaptability in the design and implementation

More Free Book



Scan to Download



Listen It

of REST APIs.

#### 4.Question

**What is the alternative approach to coding REST APIs suggested in the text?**

Answer:The text advocates for designing REST APIs rather than coding them. This involves using a configuration-driven engine that utilizes a uniform REST API layer to manage core functionalities through a structured approach, thus reducing boilerplate coding and focusing on the actual web application logic.

#### 5.Question

**How can standardization in REST API design benefit developers and organizations?**

Answer:Standardization can lead to the creation of reusable frameworks and libraries, enhancing developer productivity by simplifying coding processes. It enables better interoperability among different systems and enhances the ability to transition web applications to more scalable architectures.

**More Free Book**



Scan to Download



Listen It

## 6.Question

**What principles should guide the design of REST APIs according to the text?**

Answer:1. A REST API should be designed, not coded, allowing for a configuration-driven approach. 2. Consistency is important; uniform designs help navigate complexity and enhance usability. 3. The development process should prioritize tools that enhance productivity and allow graphical design of APIs instead of solely relying on coding.

## 7.Question

**How does the WRML approach enhance REST API design?**

Answer:The WRML approach emphasizes a conceptual framework that allows for the separation of data structure definitions from implementation, promotes the reusability of schemas and link relations, and facilitates a more consistent architecture. It encourages a design where both server and client systems can consume the same structured definitions, enabling easier collaboration and interchangeability.

**More Free Book**



Scan to Download



**Listen It**

## 8.Question

**What is the significance of HATEOAS in REST API design?**

Answer:HATEOAS (Hypermedia as the Engine of Application State) is essential because it allows clients to navigate the API dynamically through links provided in responses, reducing the need for prior knowledge of the API structure. This makes the API more user-friendly and adaptable to change.

## 9.Question

**How does moving the traditional web page templates closer to the backend impact REST API design?**

Answer:By positioning web page templates closer to the backend, it allows developers to reuse the same data structures for different purposes, minimizes duplication of efforts, and streamlines the overall architecture. This enhances flexibility and supports the diversity of client devices that may interact with the API.

## 10.Question

**What vision for the future of REST API design is**

More Free Book



Scan to Download



Listen It

## **presented in the text?**

Answer: The vision is to establish a shared and standardized methodology for REST API design that enhances developer productivity through user-friendly tools, reduces the complexity of coding, and focuses on improving business logic in web applications instead. This approach aims to create a uniformly programmable web that allows for smoother interactions and integrations across various systems.

**More Free Book**



Scan to Download



Listen It

Ad



Scan to Download



# Try Bookey App to read 1000+ summary of world best books

Unlock **1000+** Titles, **80+** Topics

New titles added every week

Brand



Leadership & Collaboration



Time Management



Relationship & Communication



Business Strategy



Creativity



Public



Money & Investing



Know Yourself



Positive Psychology

Entrepreneurship



World History



Parent-Child Communication

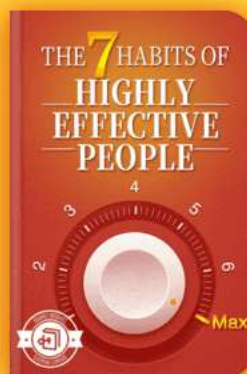
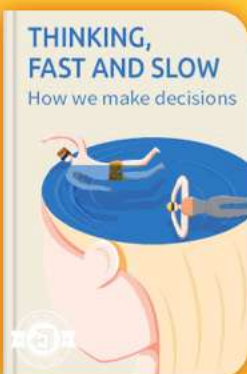


Self-care



Mind & Spirituality

## Insights of world best books



Free Trial with Bookey



## Chapter 7 | Appendix. My First REST API| Q&A

### 1.Question

**What inspired Mark Masse to design his first REST API?**

Answer:Mark Masse was inspired to design his first REST API as part of his work at Starwave, where he was tasked with developing a Java applet for NASCAR Online. The need for the applet to continuously fetch live race status data from a server, utilizing HTTP for data transmission, motivated him to create a web API that could facilitate this communication.

### 2.Question

**How did the proprietary Bulldog system contribute to the creation of the race status resource?**

Answer:The proprietary Bulldog system played a crucial role by pulling in race data from a wire feed, generating Java objects based on that data, and then publishing these objects through a new web page template. This process enabled the creation of uniquely identified plain text files hosted on a

More Free Book



Scan to Download



Listen It

standard web server, which served as the resources of the Web API needed for the Race Tracker applet.

### 3.Question

**What were the key elements of the race status resource representation designed by Mark Masse?**

Answer:The race status resource representation was formatted such that each data element appeared as a row with attributes delimited by the pipe (|) character. This included specific details such as driver information and images, all while adhering to the use of URIs for unique identification and HTTP GET requests to retrieve the current state of the race.

### 4.Question

**In what ways did Mark Masse's Race Tracker applet's API design align with REST principles?**

Answer:Even though Mark Masse's design did not strictly adhere to all REST rules, it exemplified core REST principles by utilizing URIs for resource identification, employing HTTP for communications, and allowing for

More Free Book



Scan to Download



Listen It

hypermedia controls through embedded links within the data representation.

### 5.Question

**What is the central message Mark Masse conveys about REST in this chapter?**

Answer:The central message is that REST is not a new invention, but a reflection of how the Web operates. By applying the established principles of the Web to API design, developers can naturally embrace the REST architectural style.

### 6.Question

**How does Mark Masse connect his experience with REST to its broader implications in web development?**

Answer:Mark Masse draws a parallel between his early experiences with RESTful principles and the broader practice of web development, emphasizing that designing APIs using these principles is a natural evolution for any developer familiar with the Web. It highlights the significance of leveraging existing web technologies and standards instead

More Free Book



Scan to Download



Listen It

of creating complex and convoluted systems.

**More Free Book**



Scan to Download



**Listen It**



Scan to Download



# Why Bookey is must have App for Book Lovers



## 30min Content

The deeper and clearer interpretation we provide, the better grasp of each title you have.



## Text and Audio format

Absorb knowledge even in fragmented time.



## Quiz

Check whether you have mastered what you just learned.



## And more

Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey



# Rest Api Design Rulebook Quiz and Test

[Check the Correct Answer on Bookey Website](#)

## Chapter 1 | with URIs| Quiz and Test

1.REST APIs do not use Uniform Resource

Identifiers (URIs) to identify resources on the web.

2.When designing URIs, you should include trailing slashes at the end of URIs.

3.It is recommended to use hyphens for better readability in URIs instead of underscores.

## Chapter 2 | with HTTP| Quiz and Test

1.HTTP/1.1 REST APIs integrate all features of the HyperText Transfer Protocol (HTTP/1.1).

2.The PUT method is used to retrieve a representation of a resource's state.

3.GET requests can contain headers but must not have a body.

## Chapter 3 | Quiz and Test

1.Content-Type is necessary for indicating the data type in a request/response body.

**More Free Book**



Scan to Download



[Listen It](#)

2. Custom HTTP headers are allowed to change HTTP method behavior.

3. IANA is responsible for overseeing registered media types in HTTP messages.

**More Free Book**



Scan to Download



Listen It

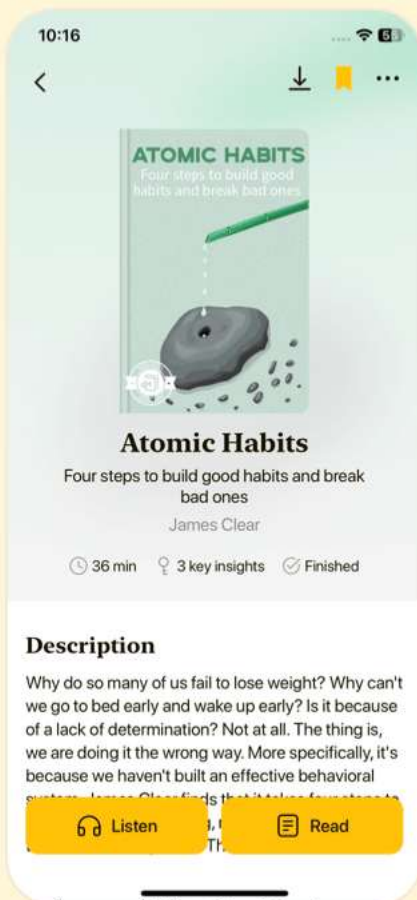


Download Bookey App to enjoy

# 1000+ Book Summaries with Quizzes

**Free Trial Available!**

Scan to Download



## Chapter 4 | Quiz and Test

- 1.JSON should be used as the primary format for resource representation in a REST API.
- 2.XML is the only format that can be used for resource representation in a REST API.
- 3.APIs must provide inconsistent forms for error responses.

## Chapter 5 | Quiz and Test

- 1.New URIs should be created for introducing new concepts in REST API design.
- 2.OAuth is not necessary for securing REST APIs since API management solutions alone provide adequate protection.
- 3.JSONP is used in JavaScript clients to enable cross-origin POST requests.

## Chapter 6 | Quiz and Test

- 1.Implementing REST APIs today poses several challenges due to the limitations of existing development tools and frameworks.
- 2.REST APIs should be coded rather than designed for a better implementation.



3. Adopting a standardized REST API design methodology can decrease operational costs by allowing for more efficient API designs.

**More Free Book**



Scan to Download



**Listen It**

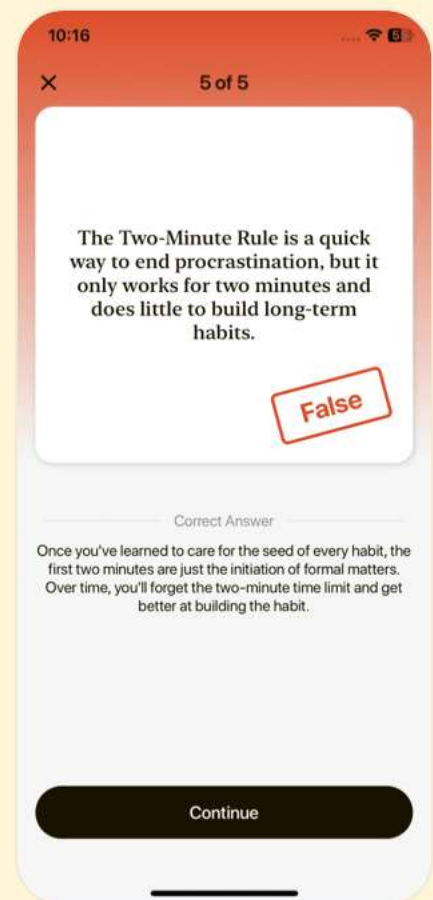
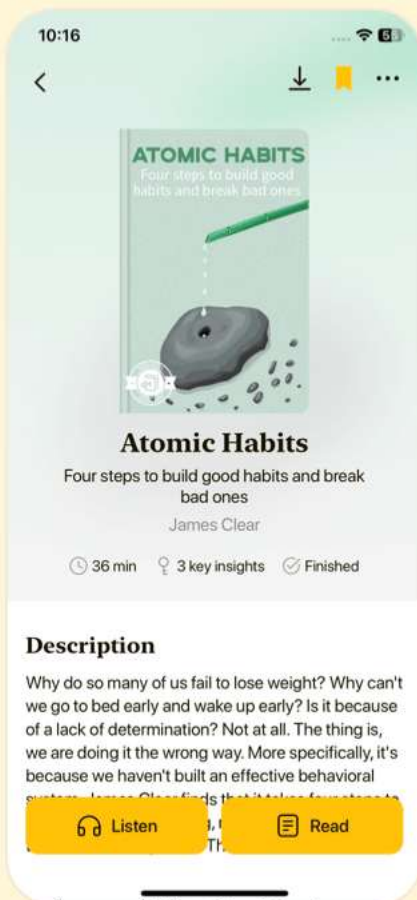


Download Bookey App to enjoy

# 1000+ Book Summaries with Quizzes

**Free Trial Available!**

Scan to Download



## Chapter 7 | Appendix. My First REST API| Quiz and Test

1. Mark Masse designed his first REST API while working for Starwave in June 1997.
2. The Race Tracker applet used HTTP POST requests to fetch resource representations.
3. The Race Tracker applet strictly followed all REST rules during its implementation.

More Free Book



Scan to Download



Listen It



Download Bookey App to enjoy

# 1000+ Book Summaries with Quizzes

**Free Trial Available!**

Scan to Download

